Cyberscope

A TAC Security Company

Audit Report

RIV

May 2026

Repository

https://github.com/GenesisTechAT/FINAL-RIV-STAKING-BEFORE-AUDIT/tree/main

Commit

1e6a1b7055c22a80893fb9add0ac2f8fee8f3483

Audited by © cyberscope

Table of Contents

Risk Classification

The criticality of findings in Cyberscope's smart contract audits is determined by evaluating multiple variables. The two primary variables are:

1. **Likelihood of Exploitation**: This considers how easily an attack can be executed, including the economic feasibility for an attacker.
2. **Impact of Exploitation**: This assesses the potential consequences of an attack, particularly in terms of the loss of funds or disruption to the contract's functionality.

Based on these variables, findings are categorized into the following severity levels:

1. **Critical**: Indicates a vulnerability that is both highly likely to be exploited and can result in significant fund loss or severe disruption. Immediate action is required to address these issues.
2. **Medium**: Refers to vulnerabilities that are either less likely to be exploited or would have a moderate impact if exploited. These issues should be addressed in due course to ensure overall contract security.
3. **Minor**: Involves vulnerabilities that are unlikely to be exploited and would have a minor impact. These findings should still be considered for resolution to maintain best practices in security.
4. **Informative**: Points out potential improvements or informational notes that do not pose an immediate risk. Addressing these can enhance the overall quality and robustness of the contract.

Severity	Likelihood / Impact of Exploitation
Critical	Highly Likely / High Impact
Medium	Less Likely / High Impact or Highly Likely/ Lower Impact
Minor / Informative	Unlikely / Low to no Impact

Review

Repository	https://github.com/GenesisTechAT/FINAL-RIV-STAKING-BEFORE-AUDIT/tree/main
Commit	1e6a1b7055c22a80893fb9add0ac2f8fee8f3483

Audit Updates

Initial Audit	29 Apr 2026

Source Files

Filename	SHA256
constants.rs	6d7f9a10c10088e47b4aca8193938d41d286c0fa24a17449b3bd186a03d8381a
errors.rs	a2ad01f9edbe0b07011b530d46750c1f126c18b83c63b5161e9a3fd8673fe83c
lib.rs	7e2965883ce3f4abd7be87fbb921502e40bb1b34b4231a9557ffe89f571c3a46
state/config.rs	4caab3c9e32b8136610964b555e836e6bc863a06777c559a08ebaeea644bb21c
state/lock.rs	d718e8c8b281d31f342f0e4a3f30d8427929120473a5ff44a7aa3b727f7e7307
state/mod.rs	00df13e9fe9c16ec40e4cae6fe381b9155528f0b5491b547e996185a37748e6a
state/tier.rs	fc4ee26e9b4bc64931f11a333e73e208bd14518d560ea70e67a565ac493f3eae

state/user.rs	a62c890851f742e5fd57f0c040da3aa742916b30589be9bdac1acd395491d755
rewards/mod.rs	02dcc7d1f4e43544adce6de0eed5f7d43affe2c06e3d688a62242610ce0eec40
instructions/begin_unlocking.rs	378d7c9c307e6d3abe195fa1ad94f857b1b9dddfdb1168d44d0a1192f99f4a81
instructions/claim_rewards.rs	ebc0b10289212acf335e753567a0fc541a068a16bf3442cd5ac4e4d391aa1f74
instructions/create_lock.rs	d075bfc609df0b4842c01abe95bb2bb7123ea5bfddc2bfee916b82239170dc21
instructions/dismantle.rs	b8f02fd37ec6d07c931d6736c3a687b1819a1ab0bb807440eef18f081b31f205
instructions/fund_rewards.rs	5b7bdd92ff954eaf82166676ec24d4eaeb85406ceffbed4c41a0e2f7a6fc7462
instructions/initialize_config.rs	6cd8de7481d4545198d306420378e8a4067d525f784a808309267df40719d530
instructions/initialize_tier.rs	29a9420a48f8e7d27454324f933789e03fd2953a73aa398bf87c5853cd9c4695
instructions/mod.rs	6291cb7bbe674ea751cc05d163feb13974184b76eb7ebfe9d2b1127d0fd08bd6
instructions/pause.rs	c6acc3a65cf2bb6354c8c1a6db189ef555d63079cc10ba980d4e27d25a0faaec
instructions/pause_rewards.rs	7d5264093f4587a2b736bacb386c6e732aafec7c9a6b0e188894bd2b5d283bbb
instructions/set_apr.rs	a0294d1efee118c550d6139ddb79f6c0314de212f776cd0e2818cce1c748b808

instructions/settle_batch.rs	40b4f1717d1865f1f88c1d50d15dde35f0e7b6ffb45ca7ee7673325d00726782
instructions/top_up.rs	cd5ec763741a209fe8031b6ec485a95f3336ecb573e9d57b4e10a598694b9c83
instructions/transfer_admin.rs	8d5169fbc28f56c7c3641df9e0c7d5a42d8c70e69b697a7f4c082f56a5d3f22c
instructions/update_config.rs	218ffe189a1bf586357937fa341ef1e5e9518db030a3e8375dd04e885d47277e
instructions/update_tier.rs	7445a2e4ac7fa32ef522a977a0f567e750315546db08066fdfaf5edbe1071bd6
instructions/withdraw_released.rs	b7c4fc67ab16dd6a5ee12956c6ab81e4792dc6eaae528a7c943ce611c28b6957

Overview

The contract implements a multi-tier SPL token staking program for the Solana blockchain, written using the Anchor framework. A single SPL token mint chosen at deployment time is staked by users into one of three tiers, each configured by an admin with its own APR in basis points, cooling period, and per-tier deposit cap. Staked tokens are held in a treasury vault PDA owned by the program, while reward tokens of the same mint are held in a separate reward vault that the admin funds via a dedicated fund_rewards instruction.

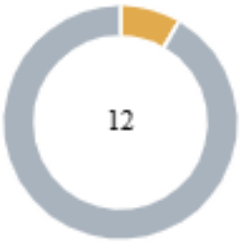
Users interact with the program by calling create_lock to open a position in a tier, top_up to add more principal to an existing active lock, and claim_rewards to retrieve accrued rewards while the lock remains open. To exit a position the user calls begin_unlocking, which transitions the lock from Active to Cooling and starts a tier-specific timer, followed by withdraw_released after the cooling period elapses to receive both principal and any remaining rewards. Each user can hold at most one Active lock per tier and a configurable number of Cooling locks per tier, with separate per-user accounts tracking aggregate stake and per-tier sequencing.

Reward accrual is tracked using a global cumulative index per tier scaled by 1e18 fixed-point arithmetic that grows at the tier's current APR over elapsed seconds. Each lock records the index value at the time of its last settlement, and accrued rewards for that lock are computed as principal multiplied by the difference between the current index and the lock's stored watermark. A bounded APR history per tier records every APR change, pause, and unpause boundary, allowing the program to reconstruct historical index values for cooling locks settled after their release timestamp. Before accepting a new deposit or APR change, the program runs a forward-looking solvency check that ensures the reward vault can cover all settled rewards, all unsettled accrual estimated as a conservative upper bound, and the projected liability over a configurable solvency horizon across every tier.

The admin holds wide-ranging operational controls including the ability to pause new deposits, pause outflows for claims and withdrawals, pause reward accrual globally, adjust per-tier APRs subject to a per-call delta cap, modify policy parameters such as caps and the solvency horizon, and transfer admin rights to a new pubkey. An emergency dismantle workflow lets the admin wind the protocol down by first pausing intake and rewards, calling begin_dismantle to snapshot total obligations, and processing each lock through dismantle_lock which is permissionless to allow third-party crank execution. Once every

lock has been processed and the global unclaimed-rewards counter reaches zero, finalize_dismantle sweeps any remaining tokens in both vaults to a recovery address fixed at deployment time, marking the protocol permanently finalized.

Findings Breakdown



Critical	0
Medium	1
Minor / Informative	11

Severity	Unresolved	Acknowledged	Resolved	Other
Critical	0	0	0	0
Medium	1	0	0	0
Minor / Informative	11	0	0	0

Diagnostics

Critical Medium Minor / Informative

Severity	Code	Description	Status
Medium	AHER	APR History Exhaustion Risk	Unresolved
Minor / Informative	AHO	APR History Ordering	Unresolved
Minor / Informative	AIOV	APR Increase Outrun Vault	Unresolved
Minor / Informative	ARLB	APR Rate Limit Bypass	Unresolved
Minor / Informative	CCR	Contract Centralization Risk	Unresolved
Minor / Informative	FSRR	Frequent Settle Rounds Rewards	Unresolved
Minor / Informative	MFANV	Mint Freeze Authority Not Validated	Unresolved
Minor / Informative	PTIB	Partial Tier Initialization Brick	Unresolved
Minor / Informative	PWF	Pausable Withdrawal Functionality	Unresolved
Minor / Informative	PUB	Possible Unlock Block	Unresolved
Minor / Informative	TSI	Token Sufficiency Insurance	Unresolved
Minor / Informative	WCB	Wallet Cap Bypass	Unresolved

AHER - APR History Exhaustion Risk

Criticality	Medium
Location	instructions/set_apr.rs#L101 instructions/pause_rewards.rs#L80 state/tier.rs#L40
Status	Unresolved

Description

The protocol stores APR change checkpoints in a bounded vector with a maximum of 64 entries enforced via `#[max_len(MAX_APR_HISTORY_LEN)]`. Each call to set_apr while rewards are paused appends a redundant `apr_bps=0` checkpoint that duplicates the regime captured by the prior pause checkpoint, and there is no on-chain mechanism to prune stale entries. Once the history reaches its 64-entry cap, set_apr, pause_rewards, and unpause_rewards all permanently fail their slot-reservation checks, leaving the admin unable to adjust APR or freeze accrual without a program upgrade. An adversarial or unintentionally aggressive admin could fill the history within roughly 63 actions, and the existing buffer logic in set_apr only delays this without preventing it.

The most damaging downstream consequence is that pause_rewards becomes unreachable once the history is full, which means rewards continue accruing at the tier's APR with no way for admin to halt them. Over time the cumulative_index grows past what reward_vault.amount can cover, and claim_rewards and withdraw_released begin reverting with RewardVaultUnderfunded for affected locks, freezing user principal and accrued rewards in place. Because begin_dismantle itself requires rewards_paused to be true as a precondition, the emergency wind-down path is also blocked, leaving the protocol in a stuck state where neither normal exits nor liquidation can proceed without a program upgrade.

```
let required_buffer: usize = if ctx.accounts.config.rewards_paused { 1
} else { 2 };
require!(
    tier.apr_history.len() + required_buffer < MAX_APR_HISTORY_LEN,
    StakingError::AprHistoryFull
);
```

Recommendation

The team is advised to enforce APR history capacity so that one checkpoint slot is always reserved for the emergency wind-down path. In particular, `set_apr` , `pause_rewards` , and `unpause_rewards` should never be able to consume the final available position in `apr_history` ; that position should remain reserved so `begin_dismantle` can atomically push the final zero-APR pause checkpoint into `apr_history` .

`begin_dismantle` should then set `rewards_paused` and append the final pause checkpoint itself, instead of depending on a prior successful `pause_rewards` call. This keeps the emergency wind-down path reachable even when normal APR-management operations have nearly exhausted the bounded vector. Together, these changes ensure the protocol can always freeze reward accrual and enter dismantle without being blocked by APR history exhaustion.

AHO - APR History Ordering

Criticality	Minor / Informative
Location	instructions/pause_rewards.rs#L80 instructions/set_apr.rs#L97
Status	Unresolved

Description

The set_apr handler enforces `require!(last.effective_at < now)` before pushing a new apr_history entry, which guarantees the vector remains strictly ascending by effective_at across APR changes. The push_pause_checkpoint and push_unpause_checkpoint helpers used by pause_rewards and unpause_rewards do not perform this check and instead push whichever Unix timestamp Clock::get returns. Solana's unix_timestamp is generally monotonic across slots but the runtime does not strictly guarantee monotonicity, and any backward drift between two pause-related transactions would produce a non-sorted apr_history. After that point, binary_search_by_key in index_at returns undefined results, silently corrupting historical reward computations for cooling-past-release locks.

```
fn push_pause_checkpoint(tier: &mut Tier, now: i64) -> Result<()> {
    require!(
        tier.apr_history.len() + 1 < MAX_APR_HISTORY_LEN,
        StakingError::AprHistoryFull
    );
    tier.apr_history.push(AprCheckpoint {
        effective_at: now,
        cumulative_index: tier.cumulative_index,
        apr_bps: 0,
    });
    Ok(())
}
```

Recommendation

The team is advised to add a monotonicity check inside both push_pause_checkpoint and push_unpause_checkpoint that mirrors the guard already present in set_apr, for example `require!(last.effective_at <= now, StakingError::AprChangeTooSoon)` using non-strict less-than-or-equal to permit legitimate same-second pushes. This will close the gap between the two existing pathways that mutate apr_history and ensure the binary_search invariant in index_at holds independently of any future weakening of Solana's clock guarantees.

AIOV - APR Increase Outrun Vault

Criticality	Minor / Informative
Location	instructions/set_apr.rs#L78
Status	Unresolved

Description

The set_apr instruction performs a solvency check that verifies the reward vault covers all settled and unsettled rewards plus projected accrual at the new APR over solvency_horizon_seconds, but this check is bounded to that horizon. An APR change that passes the check guarantees solvency only for the duration of the horizon, and only if no admin or user activity is required to refresh the projection during that window. If the protocol sits idle beyond the horizon, actual accrued rewards continue to grow at the new and potentially much higher APR, eventually exceeding what the reward_vault can cover. An APR increase accelerates the rate at which actual accrual outpaces the vault.

```
let mut target =
rewards::TierSolvencyData::from_tier(&ctx.accounts.tier, now,
rewards_paused)?;
target.apr_bps = new_apr_bps;
rewards::check_solvency_with_error(
    ctx.accounts.reward_vault.amount,
    config.total_unclaimed_rewards,
    config.solvency_horizon_seconds,
    target,
    0,
    &other_tiers,
    StakingError::InsufficientRewardRunwayAtNewApr,
)?;
```

Recommendation

The team is advised to consider mechanisms that keep solvency validation alive beyond the configured horizon, such as a permissionless crank instruction that any caller can invoke to re-run the solvency check and either succeed silently or auto-pause rewards if the vault has fallen below the threshold, or an automatic check inside frequently called user instructions that pauses accrual when projected obligation exceeds vault balance.

ARLB - APR Rate Limit Bypass

Criticality	Minor / Informative
Location	instructions/set_apr.rs#L65 instructions/set_apr.rs#L97 constants.rs#L12
Status	Unresolved

Description

The MAX_APR_DELTA_BPS constant restricts each set_apr call to at most a 500 bps change, which appears intended to give users predictable rate movement and time to react via begin_unlocking if they dislike the new APR. However the only timing guard is `require!(last.effective_at < now)`, which merely blocks two changes within the same Unix second. An admin can chain three or more set_apr calls across consecutive Solana slots and shift the APR by 1500 bps or more in roughly one second, defeating the stated protection. Users staking under one APR have no realistic chance to react before the rate has moved arbitrarily far from the value they originally observed.

```
require!(
    new_apr_bps.abs_diff(ctx.accounts.tier.apr_bps) <=
MAX_APR_DELTA_BPS,
    StakingError::AprChangeTooLarge
);
...
if let Some(last) = tier.apr_history.last() {
    require!(last.effective_at < now, StakingError::AprChangeTooSoon);
}
```

Recommendation

The team is advised to introduce a minimum interval between APR changes by replacing the same-slot guard with `require!(last.effective_at + MIN_APR_CHANGE_INTERVAL <= now)` where MIN_APR_CHANGE_INTERVAL is a constant chosen to match the protocol's commitment to user reaction time, typically one hour or one day. This will ensure that the per-call delta cap translates into a meaningful per-time-window cap and aligns the on-chain enforcement with the rate-of-change protection users expect when staking under a published APR.

CCR - Contract Centralization Risk

Criticality	Minor / Informative
Location	instructions/set_apr.rs instructions/dismantle.rs instructions/initialize_config.rs instructions/initialize_tier.rs instructions/pause_rewards.rs instructions/pause.rs instructions/update_config.rs instructions/update_tier.rs
Status	Unresolved

Description

The contract's functionality and behavior are heavily dependent on external parameters or configurations. While external configuration can offer flexibility, it also poses several centralization risks that warrant attention. Centralization risks arising from the dependence on external configuration include Single Point of Control, Vulnerability to Attacks, Operational Delays, Trust Dependencies, and Decentralization Erosion.

Recommendation

To address this finding and mitigate centralization risks, it is recommended to evaluate the feasibility of migrating critical configurations and functionality into the contract's codebase itself. This approach would reduce external dependencies and enhance the contract's self-sufficiency. It is essential to carefully weigh the trade-offs between external configuration flexibility and the risks associated with centralization.

FSRR - Frequent Settle Rounds Rewards

Criticality	Minor / Informative
Location	rewards/mod.rs#L94 instructions/settle_batch.rs#L36
Status	Unresolved

Description

The settle_lock function computes accrued as `principal × delta / INDEX_SCALE` using integer division, which floors the result toward zero whenever the product is smaller than 1e18. When settle_batch is invoked frequently with short intervals between calls, each per-call delta becomes small enough that the per-call accrued rounds to zero for locks with modest principals, yet lock.last_reward_index is still advanced to upper_index regardless of the rounded value. The user therefore loses the per-call fraction permanently because future settles compute delta from the new advanced watermark rather than the original one, and the rounded-off rewards do not appear anywhere in the protocol's user-visible accounting since they leave the unsettled_upper bucket without entering total_unclaimed_rewards. Because settle_batch is permissionless, an attacker can grief specific users by repeatedly settling their locks at high frequency, causing a fraction of their accrued rewards to be silently absorbed into the reward_vault surplus that is ultimately swept to recovery on dismantle.

```
let accrued = u64::try_from(
    (lock.principal as u128)
        .checked_mul(delta)
        .and_then(|v| v.checked_div(INDEX_SCALE))
        .ok_or(StakingError::MathOverflow)?,
)
.map_err(|_| error!(StakingError::MathOverflow))?;

lock.unclaimed_rewards = lock
    .unclaimed_rewards
    .checked_add(accrued)
    .ok_or(StakingError::MathOverflow)?;
lock.last_reward_index = upper_index;
```

Recommendation

The team is advised to make settle_batch a permissioned function that could also enforce a minimum interval between successive settles for the same lock to make the grief-frequency attack uneconomical.

MFANV - Mint Freeze Authority Not Validated

Criticality	Minor / Informative
Location	instructions/initialize_config.rs#L25,111
Status	Unresolved

Description

The initialize_config handler accepts any SPL Token mint without inspecting its freeze_authority field. If the mint has an active freeze authority, that authority can subsequently freeze either the treasury_vault or reward_vault token account PDAs, which would block all token::transfer calls flowing through this protocol and effectively brick user withdrawals and reward claims. The protocol implicitly trusts that the mint chosen at deployment will not be used adversarially against its vault accounts, but this trust assumption is not enforced or surfaced anywhere in the contract.

```
pub mint: Account<'info, Mint>,
...
config.mint = ctx.accounts.mint.key();
```

Recommendation

The team is advised to add a hard require on `mint.freeze_authority` being None at initialization. This removes the manual verification steps and surfaces the assumption directly in the contract code.

PTIB - Partial Tier Initialization Brick

Criticality	Minor / Informative
Location	instructions/pause_rewards.rs#L21 instructions/dismantle.rs#L79 instructions/dismantle.rs#L451 instructions/update_config.rs#L96 rewards/mod.rs#L221
Status	Unresolved

Description

The protocol assumes all three tiers indexed 0, 1, and 2 are initialized before any non-init operation runs. The pause_rewards and unpause_rewards handlers declare tier_0, tier_1, and tier_2 as named accounts in their context struct, so any attempt to invoke them before all three tiers exist fails at the Anchor account-validation stage with an account-not-found error. The same applies to begin_dismantle, finalize_dismantle, set_solvency_horizon, and every user-facing instruction that goes through collect_other_tier_data such as create_lock, top_up, and set_apr, each of which expects exactly the right number of tier accounts in remaining_accounts. If admin completes initialize_config but only initializes one or two tiers before opening the protocol, users can still claim and withdraw existing locks but cannot create new ones, admin loses access to the global pause_rewards lever entirely, and the dismantle path is unreachable until the missing tiers are initialized.

```
#[account(mut, seeds = [TIER_SEED, &[0u8]], bump = tier_0.bump)]
pub tier_0: Account<'info, Tier>,

#[account(mut, seeds = [TIER_SEED, &[1u8]], bump = tier_1.bump)]
pub tier_1: Account<'info, Tier>,

#[account(mut, seeds = [TIER_SEED, &[2u8]], bump = tier_2.bump)]
pub tier_2: Account<'info, Tier>,
```

Recommendation

The team is advised to gate user-facing instructions and pause/dismantle handlers behind a Config flag set only after all three tiers have been initialized, so the protocol cannot enter operational state in a partially configured form. This approach removes the foot-gun where partial initialization silently produces a half-functional protocol.

PWF - Pausable Withdrawal Functionality

Criticality	Minor / Informative
Location	instructions/pause.rs#L52 instructions/claim_rewards.rs#L78 instructions/withdraw_released.rs#L110
Status	Unresolved

Description

The handle_pause_outflow handler sets config.outflow_paused to true and there is no on-chain mechanism that forces admin to unpause within any time bound. Because claim_rewards and withdraw_released both gate on this flag, admin can pause outflow at any moment and leave the flag set indefinitely, freezing user access to both accrued rewards and previously cooling principal even after the lock's release_timestamp has elapsed.

```
pub fn handle_pause_outflow(ctx: Context<TogglePause>) -> Result<()> {
    ctx.accounts.config.require_admin_controls_allowed()?;
    ctx.accounts.config.outflow_paused = true;
    ...
}
```

Recommendation

The team should carefully manage the private keys of the owner’s account. We strongly recommend a powerful security mechanism that will prevent a single user from accessing the contract admin functions.

These measurements do not decrease the severity of the finding

- Introduce a time-locker mechanism with a reasonable delay.
- Introduce a multi-signature wallet so that many addresses will confirm the action.
- Introduce a governance model where users will vote about the actions.

PUB - Possible Unlock Block

Criticality	Minor / Informative
Location	instructions/update_config.rs#L59 instructions/begin_unlocking.rs#L33
Status	Unresolved

Description

The handle_set_max_cooling_locks instruction lets admin lower the per-user-per-tier cooling lock cap with no consideration for users who already hold cooling locks above the new cap. Users whose cooling_lock_count is at or above the new cap can no longer call begin_unlocking on any of their active locks in that tier until enough existing cooling locks finish their cooling period and are withdrawn via withdraw_released.

```
pub fn handle_set_max_cooling_locks(ctx: Context<UpdateConfig>, count:
u16) -> Result<()> {
    ctx.accounts.config.require_admin_controls_allowed()?;
    require!(count > 0, StakingError::ZeroAmount);
    let old = ctx.accounts.config.max_cooling_locks_per_user;
    ctx.accounts.config.max_cooling_locks_per_user = count;
    ...
}
```

Recommendation

The team is advised to consider requiring count to be greater than or equal to the previous value so the cap can only loosen and not tighten.

TSI - Token Sufficiency Insurance

Criticality	Minor / Informative
Location	instructions/claim_rewards.rs#L94 instructions/withdraw_released.rs#L136 instructions/dismantle.rs#L116
Status	Unresolved

Description

The protocol provides no graceful-degradation path for the case in which reward_vault.amount falls below outstanding reward obligations. Whenever the vault becomes insufficient relative to what users are owed, the claim_rewards and withdraw_released instructions begin to revert with RewardVaultUnderfunded for affected locks, leaving users unable to retrieve their accrued rewards or principal through the normal exit paths even though their principal sits untouched in the treasury vault. The dismantle workflow is similarly compromised because begin_dismantle requires reward_vault.amount to cover total_unclaimed_rewards plus total_unsettled_upper across all tiers, a condition that fails once accrual has outpaced the vault, blocking the wind-down route entirely. With both ordinary withdrawals and the emergency exit blocked, the protocol enters a stuck state in which only an off-chain admin top-up via fund_rewards or a program upgrade can restore functionality, and an under-resourced or unresponsive admin would leave user funds frozen indefinitely. An example condition that can produce this state is described in the `AIOV` .

```
require!(
    ctx.accounts.reward_vault.amount >= claim_amount,
    StakingError::RewardVaultUnderfunded
);
...
require!(
    (ctx.accounts.reward_vault.amount as u128) >=
total_reward_obligation,
    StakingError::RewardVaultUnderfundedForDismantle
);
```

Recommendation

The team is advised to add graceful-degradation paths so that an underfunded reward vault does not hold user principal hostage.

WCB - Wallet Cap Bypass

Criticality	Minor / Informative
Location	state/user.rs#L4 instructions/create_lock.rs#L179 state/config.rs#L31
Status	Unresolved

Description

The per_wallet_cap is enforced against config.per_wallet_cap by reading user_state.total_staked, where UserState is a PDA seeded by the user's pubkey. A user wishing to stake more than per_wallet_cap can simply create additional Solana wallets and stake independently from each, since each wallet has its own UserState account and the protocol has no mechanism to associate distinct pubkeys with the same human or beneficiary. The cap therefore operates as a soft guard against accidental over-concentration in a single key but does not constitute a meaningful limit on the aggregate stake a single off-chain entity can place in the protocol, and any threat model that relies on per_wallet_cap to prevent concentration of risk against a single beneficiary is unsound.

```
if config.per_wallet_cap > 0 {
    require!(
        ctx.accounts
            .user_state
            .total_staked
            .checked_add(amount)
            .ok_or(StakingError::MathOverflow)?
            <= config.per_wallet_cap,
        StakingError::PerWalletCapExceeded
    );
}
```

Recommendation

The team is advised to document explicitly that per_wallet_cap is a per-key cap rather than a per-entity cap, and to clarify whether the protocol's intended stake-concentration controls are enforced at the contract layer or expected to be enforced through off-chain processes.

Summary

RIV contract implements a staking mechanism. This audit investigates security issues, business logic concerns and potential improvements.

Disclaimer

The information provided in this report does not constitute investment, financial or trading advice and you should not treat any of the document's content as such. This report may not be transmitted, disclosed, referred to or relied upon by any person for any purposes nor may copies be delivered to any other person other than the Company without Cyberscope's prior written consent. This report is not nor should be considered an "endorsement" or "disapproval" of any particular project or team. This report is not nor should be regarded as an indication of the economics or value of any "product" or "asset" created by any team or project that contracts Cyberscope to perform a security assessment. This document does not provide any warranty or guarantee regarding the absolute bug-free nature of the technology analyzed, nor do they provide any indication of the technologies proprietors' business, business model or legal compliance. This report should not be used in any way to make decisions around investment or involvement with any particular project. This report represents an extensive assessment process intending to help our customers increase the quality of their code while reducing the high level of risk presented by cryptographic tokens and blockchain technology.

Blockchain technology and cryptographic assets present a high level of ongoing risk Cyberscope's position is that each company and individual are responsible for their own due diligence and continuous security Cyberscope's goal is to help reduce the attack vectors and the high level of variance associated with utilizing new and consistently changing technologies and in no way claims any guarantee of security or functionality of the technology we agree to analyze. The assessment services provided by Cyberscope are subject to dependencies and are under continuing development. You agree that your access and/or use including but not limited to any services reports and materials will be at your sole risk on an as-is where-is and as-available basis Cryptographic tokens are emergent technologies and carry with them high levels of technical risk and uncertainty. The assessment reports could include false positives false negatives and other unpredictable results. The services may access and depend upon multiple layers of third parties.

About Cyberscope

Cyberscope is a TAC blockchain cybersecurity company that was founded with the vision to make web3.0 a safer place for investors and developers. Since its launch, it has worked with thousands of projects and is estimated to have secured tens of millions of investors' funds.

Cyberscope is one of the leading smart contract audit firms in the crypto space and has built a high-profile network of clients and partners.

A **TAC Security** Company

The Cyberscope team

cyberscope.io